

Practica - 8 Bucles y Declaraciones Condicionales

Luis Gálvez

Tabla de contenidos

1 Proposito de la practica	1
1.1 Ejercicio 1:	1
1.1.1 Desarrollo	2
1.2 Ejercicio 2: Parseo de coordenadas y WKT (Volcán Galeras) . . .	3
1.2.1 Desarrollo	4
1.3 Preguntas	5
1.3.1 Desarrollo	5

1 Proposito de la practica

1.1 Ejercicio 1:

Contexto: Estás trabajando en un proyecto de zonificación agrícola en el Eje Cafetero. Tienes una lista de fincas con su respectiva altitud. Sabiendo que el café de especialidad de la región requiere alturas entre los 1700 y los 2200 metros sobre el nivel del mar (msnm), necesitas crear un algoritmo que clasifique automáticamente cada finca.

Instrucciones de código:

1. Crea una colección (lista o vector) llamada `altitudes_fincas` que contenga los siguientes valores numéricos: 1350, 1750, 1900, 2300, 1600, 2100.
2. Inicializa dos contadores en cero: `conteo_especial` y `conteo_tradicional`.
3. Inicializa dos colecciones vacías (listas o vectores) llamadas `fincas_especiales` y `fincas_tradicionales`.
4. Construye un ciclo `for` que recorra uno a uno los valores de `altitudes_fincas`.
5. Dentro del ciclo, utiliza una declaración `if - else` con operadores lógicos (AND) para evaluar cada altitud:

- **Si** la altitud es mayor o igual a 1700 **Y** menor o igual a 2200, agrega la altitud a la colección `fincas_especiales` y suma 1 al `conteo_especial`.
 - **De lo contrario**, agrégala a la colección `fincas_tradicionales` y suma 1 al `conteo_tradicional`.
6. Fuera del ciclo, imprime un reporte final que muestre cuántas fincas son aptas para café de especialidad y cuántas para café tradicional, mostrando además los valores guardados en ambas colecciones.

1.1.1 Desarrollo

```
# Lista de altitudes de las fincas
altitudes_fincas = [1350, 1750, 1900, 2300, 1600, 2100]

#-----
# Inicializar contadores
conteo_especial = 0
conteo_tradicional = 0

#-----
# Inicializar listas vacías
fincas_especiales = []
fincas_tradicionales = []

#-----
# Recorrer las altitudes con un ciclo for
for altitud in altitudes_fincas:

    # Evalua si la finca es apta para café de especialidad
    if altitud >= 1700 and altitud <= 2200:
        fincas_especiales.append(altitud)
        conteo_especial += 1

    # Si no cumple la condición
    else:
        fincas_tradicionales.append(altitud)
        conteo_tradicional += 1

#-----
# Imprimir reporte final
print("===== REPORTE DE CLASIFICACIÓN =====")

print("\nFincas aptas para café de especialidad:")
print(f"Cantidad: {conteo_especial}")
print(f"Altitudes: {fincas_especiales}")
```

```
print("\nFincas aptas para café tradicional:")
print(f"Cantidad: {conteo_tradicional}")
print(f"Altitudes: {fincas_tradicionales}")
```

===== REPORTE DE CLASIFICACIÓN =====

Fincas aptas para café de especialidad:

Cantidad: 3

Altitudes: [1750, 1900, 2100]

Fincas aptas para café tradicional:

Cantidad: 3

Altitudes: [1350, 2300, 1600]

1.2 Ejercicio 2: Parseo de coordenadas y WKT (Volcán Galeras)

Contexto: El IDEAM te ha encargado programar el software de una estación hidrológica automatizada en el Río Magdalena. El sensor debe tomar lecturas continuas del nivel del agua. Si el río supera los 8.5 metros, el sistema debe disparar una alerta de inundación y detener las mediciones para proteger el equipo. Si después de cierto número de lecturas todo está normal, el sensor entra en modo de reposo.

Instrucciones de código:

1. Define las siguientes variables de configuración:
 - `nivel_alerta = 8.5`
 - `max_lecturas = 12`
 - `lectura_actual = 0`
 - Una bandera lógica (Booleana) llamada `alerta_inundacion` inicializada como Falsa (`False` / `FALSE` / `false`).
2. Construye un ciclo **while** que se ejecute **mientras** `lectura_actual` sea menor a `max_lecturas`.
3. Dentro del ciclo, simula la medición del río generando un número decimal aleatorio entre 5.0 y 9.0 (usa la función nativa de tu lenguaje para números aleatorios, ej. `random.uniform` en Python, `runif` en R, o `rand` en Julia).
4. Incrementa en 1 la variable `lectura_actual`. Imprime en pantalla el número del intento y el nivel del agua medido (con dos decimales).
5. Usa un condicional **if** para verificar si la medición superó o igualó el `nivel_alerta`. Si es así:
 - Cambia la bandera `alerta_inundacion` a Verdadera.
 - Imprime un mensaje de “¡ALERTA ROJA! Posible desbordamiento.”

- Rompe el ciclo inmediatamente usando el comando adecuado (`break`).
6. Fuera del ciclo `while`, utiliza un condicional para evaluar tu bandera lógica. Si la bandera sigue siendo Falsa, imprime: “Monitoreo finalizado. Niveles estables.”

1.2.1 Desarrollo

```
# Importar librería para números aleatorios
import random

#-----
# Variables de configuración
nivel_alerta = 8.5
max_lecturas = 12
lectura_actual = 0
alerta_inundacion = False

#-----
# Ciclo while para monitoreo continuo
while lectura_actual < max_lecturas:

    # Genera la medición aleatoria entre 5.0 y 9.0
    nivel_rio = random.uniform(5.0, 9.0)

    # Incrementar el contador de lecturas
    lectura_actual += 1

    # Mostrar resultado de la medición
    print(
        f"Lectura {lectura_actual}: "
        f"Nivel del río = {nivel_rio:.2f} m"
    )

    # Verificar condición de alerta
    if nivel_rio >= nivel_alerta:

        alerta_inundacion = True

        print(";ALERTA ROJA! Posible desbordamiento.")

        # Romper el ciclo inmediatamente
        break

# Evaluar estado final del monitoreo
```

```
if not alerta_inundacion:
    print("\nMonitoreo finalizado. Niveles estables.")
```

Lectura 1: Nivel del río = 6.10 m
Lectura 2: Nivel del río = 8.34 m
Lectura 3: Nivel del río = 8.98 m
¡ALERTA ROJA! Posible desbordamiento.

1.3 Preguntas

1. **Sobre el Ejercicio 1 (Indentación y Bloques):** Si estuvieras desarrollando el código en **Python**, ¿cómo sabe el computador qué instrucciones pertenecen dentro del ciclo **for**, cuáles pertenecen al **if**, y cuáles deben ejecutarse solo al final del programa? ¿Qué diferencia estructural existe frente a cómo lo resolverías en **R** o **Julia**?
2. **Sobre el Ejercicio 2 (Ciclos Infinitos):** En el ciclo **while** de la simulación del río, ¿qué sucedería exactamente con la memoria y la ejecución del programa si olvidas incluir la instrucción que incrementa la variable `lectura_actual` en 1? ¿Por qué este riesgo no existe al usar un ciclo **for**?
3. **Pregunta General (Operadores Lógicos):** Si estás programando en **R** o **Julia**, explica detalladamente por qué es peligroso usar el operador doble (**&&**) cuando intentas filtrar un vector completo de altitudes, y por qué en ese caso específico debes usar el operador sencillo (**&** o **.&**).

1.3.1 Desarrollo

1. Al usar ciclos **for**, operadores como **while** o condicionales como **if** se produce un espaciado en las siguientes líneas, este espaciado se conoce como indentación, y es con él que la máquina reconoce lo que está dentro del operador. En **R** o **Julia** la estructura no depende exclusivamente de los espacios, sino de delimitadores explícitos, en **R** normalmente se usan llaves **{}** para encerrar bloques, mientras que en **Julia** se cierran con “end”
2. El ciclo depende directamente del valor de la variable, siempre que esta sea menor que el máximo de lecturas se ejecutará el operador, por lo que si no incrementa queda atrapado en un bucle infinito y se colapsa. Este riesgo no existe en el ciclo **for** por que este recorre automáticamente la variable sin depender de incrementos, por lo que se elimina la posibilidad de que itere de manera infinita.
3. En **R** y **Julia**, el operador doble (**&&** en **R** y **&&** en **Julia**) está diseñado para evaluar únicamente una sola condición lógica escalar, no un vector completo. Eso significa que cuando aplicas **&&** sobre un conjunto de altitudes, el lenguaje toma solamente el primer elemento del vector y descarta los demás. Para filtrar un vector completo debes usar los

operadores vectorizados `&` en R, y `.&` en Julia. Estos operadores aplican la condición elemento por elemento. Así, cada altitud se compara individualmente con ambos límites y el resultado final es otro vector lógico completo.